

Attacking Poseidon via Graeffe-Based Root-Finding over NTT-Friendly Fields^{*}

Antonio Sanso¹ and Giuseppe Vitto²

¹ Ethereum Foundation `name.surname@ethereum.org`

² Zircuit `name@zircuit.com`

Abstract. This paper explores the algebraic structure of the POSEIDON and POSEIDON2 permutations over NTT-friendly finite fields, with a focus on preimage recovery via root-finding techniques. We introduce an algorithm for efficiently identifying single roots of high-degree univariate polynomials that emerge from these constructions, based on the *Graeffe transform* and the *tangent Graeffe method*. Our approach is evaluated on reduced-round bounty instances of these permutations at various security levels, as proposed by the Ethereum Foundation, demonstrating practical effectiveness. These results yield new insights into the security of permutation-based cryptographic primitives instantiated over NTT-friendly prime fields.

Keywords: Poseidon · Poseidon2 · Cryptanalysis · Root-finding · Graeffe · Interpolation · CICO · Zero-Knowledge · Hash Break

1 Introduction

POSEIDON [4] is a family of cryptographic permutations specifically designed for arithmetization and optimized for use in zero-knowledge proof systems. These permutations operate over finite fields $\mathbb{F}_p$ and are defined as mappings on $\mathbb{F}_p^t$. The construction of POSEIDON follows a Substitution-Permutation Network (SPN) paradigm, where the nonlinear layer consists of parallel applications of low-degree power maps of the form $x \mapsto x^d$, with $\gcd(d, p-1) = 1$. The design of POSEIDON incorporates the HADES strategy [6], which interleaves full and partial rounds to balance efficiency and security. Specifically, full rounds apply the S-box to every element in the state, while partial rounds apply it to only one element, reducing computational overhead while still increasing algebraic complexity. The full structure consists of $\frac{R_F}{2}$ full rounds, followed by R_P partial rounds, and another $\frac{R_F}{2}$ full rounds, where R_F and R_P denote the number of full and partial rounds, respectively. Full rounds primarily provide resistance against statistical attacks, while partial rounds serve to raise the algebraic degree, mitigating algebraic attacks. An extension of the original design, POSEIDON2 [5], introduces an initial linear layer and employs optimized matrices, yielding improved performance in both theoretical and practical scenarios.

The attack surface of POSEIDON has been examined in recent cryptanalytic surveys [7], which outline the balance between statistical and algebraic vulnerabilities inherent to its design. Full rounds, by applying the S-box uniformly, effectively mitigate statistical distinguishers and enhance diffusion. However, partial rounds, which apply the nonlinear transformation more selectively, create structural nuances that may be exploited by algebraic attacks due to regions of lower non-linearity within the state. The growth of the algebraic degree across the permutation is crucial in defending against attacks leveraging low-degree polynomial representations. While the use of low-degree power maps enables efficient computation, it requires careful round structuring to maintain resistance against algebraic cryptanalysis. Furthermore, the design and optimization of linear layers, including the MDS matrices, play a key role in maximizing diffusion and minimizing exploitable weaknesses.

^{*} This manuscript is Work in Progress.

Some reduced-round instances of POSEIDON have been successfully attacked using algebraic techniques, specifically by solving instances of the CICO-1 (Constrained Input - Constrained Output) problem, as demonstrated in [2]. This highlights the importance of careful parameter selection and the sensitivity of the design to the number and structure of rounds.

Our Contribution In this paper, we present a new type of algebraic attack targeting reduced-round instances of POSEIDON and POSEIDON2 over NTT-friendly prime fields. By exploiting the structure of these fields and employing the Graeffe transform, our method achieves improved efficiency in the root-finding stage of interpolation attacks. This results in a streamlined algorithm for preimage recovery that outperforms existing approaches and reveals practical vulnerabilities in these permutation-based primitives when instantiated over NTT-friendly primes.

Outline This paper is organized as follows. In Section 2, we provide additional details on the POSEIDON and POSEIDON2 permutations and survey relevant cryptanalytic attacks. Section 3 presents the core strategy of the attack in the setting of generic prime fields. In Section 4, we introduce the Graeffe Transform and Graeffe Tangent root-finding method. Finally, in Section 5, we demonstrate how to employ them to efficiently find roots of polynomials defined over NTT-friendly prime fields, and we apply them to break reduced-round instances of POSEIDON2 over 64-bit fields and POSEIDON over 256-bit fields.

Notation We write $M(d)$ to denote the time complexity of multiplying two polynomials in $\mathbb{F}_p[x]$ of degree less than d . It is known that $M(d) \in \mathcal{O}(d \log d \log \log d)$ using fast multiplication algorithms.

2 The Poseidon Permutation

The POSEIDON permutation is a cryptographic primitive specifically designed for efficient implementation in finite fields, particularly within zero-knowledge proof systems and other proof-friendly cryptographic protocols. Its design strikes a careful balance between algebraic simplicity and resistance to cryptanalytic attacks. Unlike traditional permutations used in block ciphers (e.g., AES), POSEIDON is optimized for settings where arithmetic over large prime fields is dominant and expensive operations like bitwise XOR or substitution tables are infeasible. Formally, POSEIDON defines a permutation $P : \mathbb{F}_p^T \rightarrow \mathbb{F}_p^T$ over a vector of T field elements, constructed through a sequence of carefully structured rounds. These rounds are divided into two types: *full rounds* and *partial rounds*, each comprising three essential transformation steps:

- R_F : Number of **full rounds**, applied symmetrically at the beginning and end of the permutation ($R_F/2$ each).
- R_P : Number of **partial rounds**, applied in between the full rounds.

Each round executes the following sequence of transformations:

1. **Add Round Constants.** A round-dependent constant vector $C^{(r)} = (c_0^{(r)}, c_1^{(r)}, \dots, c_{T-1}^{(r)}) \in \mathbb{F}_p^T$ is added component-wise to the current state vector $w^{(r)} = (w_0^{(r)}, w_1^{(r)}, \dots, w_{T-1}^{(r)})$, resulting in:

$$a \leftarrow w^{(r)} + C^{(r)}.$$

2. **S-Box Layer.** A non-linear transformation, given by raising state elements to a fixed power d , is applied. During a full round, this operation is applied independently to every element:

$$b \leftarrow (a_0^d, a_1^d, \dots, a_{T-1}^d).$$

Conversely, during a partial round, the operation is performed only on the first element of the state:

$$b \leftarrow (a_0^d, a_1, \dots, a_{T-1}).$$

3. **Mixing Layer.** The state vector is updated by applying a linear transformation, represented as multiplication by an MDS (Maximum Distance Separable) matrix $M \in \mathbb{F}_p^{T \times T}$:

$$w^{(r+1)} \leftarrow M \cdot b.$$

The complete permutation thus follows this structure:

$$\underbrace{\text{Full Rounds}}_{R_F/2} \rightarrow \underbrace{\text{Partial Rounds}}_{R_P} \rightarrow \underbrace{\text{Full Rounds}}_{R_F/2}.$$

The cryptographic strength of the POSEIDON permutation hinges on the careful choice of its design parameters: the round constants $C^{(r)}$, the S-Box exponent d , the MDS matrix M , and the round numbers R_F and R_P . These parameters are selected to ensure strong diffusion, non-linearity, and resistance to differential and linear cryptanalysis, while still being efficient in proof systems that rely on arithmetic circuits.

Poseidon2. POSEIDON2 is a refined version of the original POSEIDON permutation, defined over the same family of prime fields $\mathbb{F}_p$. It supports usage in both the sponge construction and the feed-forward (compression) mode, enhancing its versatility in cryptographic protocols. The core structure remains round-based, but several important changes distinguish POSEIDON2:

- **Pre-round Linear Layer.** An additional linear layer M_ϵ is applied to the state before the first round. This strengthens resistance against algebraic attacks that attempt to bypass the first rounds.
- **Non-MDS Matrices.** Unlike POSEIDON, which uses an MDS matrix with maximal branch number for each round, POSEIDON2 employs two different non-MDS matrices M_ϵ and M_I in the external and internal rounds, respectively. These matrices have a reduced branch number (less than $T - 1$), offering performance benefits.
- **Localized Round Constants.** In internal rounds, round constants are not added to every element of the state. Instead, they are applied only at the position of the S-box application (typically the first element), simplifying the implementation and reducing the number of constraints in proof systems.

These changes provide better efficiency in circuit representations and improved resilience against certain classes of algebraic and statistical attacks, while maintaining cryptographic robustness in relevant security models.

3 Attack Strategy

In this section, we present the core methodology underlying our analysis of the POSEIDON and POSEIDON2 permutations over NTT-friendly prime fields. Our focus lies on algebraic techniques that leverage the inherent polynomial structure of the permutation to recover inputs from constrained outputs, a fundamental step in evaluating the permutation’s resistance to preimage and related attacks.

Specifically, we frame our approach in terms of the *Constrained Input – Constrained Output* (CICO) problem. Given a permutation $P : \mathbb{F}_p^T \rightarrow \mathbb{F}_p^T$, the CICO problem asks us to find pairs $(w, v) \in \mathbb{F}_p^T \times \mathbb{F}_p^T$ such that $P(w) = v$, while certain coordinates of w and v are fixed according to predetermined constraints. This problem naturally arises in cryptanalysis scenarios where partial information about inputs and outputs is known or controlled.

By exploiting the algebraic structure of P , we reduce the CICO problem to finding roots of certain univariate polynomials over $\mathbb{F}_p$ derived from the permutation’s internal relations. Our attack strategy centers on efficiently identifying these roots, which correspond to candidate inputs that satisfy the given constraints. This approach enables reconstruction of the corresponding outputs and facilitates preimage recovery in the constrained setting.

3.1 Solving Univariate Systems

We solve the CICO problem for reduced-round instances of the POSEIDON permutation through the following stages:

1. **Univariate Transformation.** We restrict the permutation's input by fixing all non-constrained input entries, except one, to random values r_i . This transforms the output entries into evaluations of univariate polynomials $f_j(x) \in \mathbb{F}_p[x]$, depending only on the single unknown input state $x \in \mathbb{F}_p$. An upper bound for each polynomial's degree, $\deg(f_j)$, can be derived from the permutation's internal structure and parameters. More specifically, if d is the degree of the S-Box's non-linear power mapping, then $\deg(f_j) \leq d^{R_F+R_T}$.
2. **Interpolation.** We evaluate the univariate transformation of the permutation, as defined in the previous step, at $\deg(f_j)+1$ consecutive powers of a suitably chosen root of unity in $\mathbb{F}_p$. We then collect the outputs corresponding to the entries constrained by the CICO problem and apply an inverse Number Theoretic Transform (NTT) to interpolate the polynomials $f_j(x)$. Note that, to solve a given CICO problem, it suffices to interpolate only those polynomials $f_j(x)$ corresponding to the indices j for which the output entries $\{w_j\}_j$ are constrained.
3. **Root-Finding.** When certain output entries $\{w_j\}_j$ are constrained by the CICO problem, local solutions can be found by solving the polynomial equations $f_j(x) = w_j$. By Fermat's Little Theorem, all elements of $\mathbb{F}_p$ are roots of the polynomial $\pi_p(x) = x^p - x \in \mathbb{F}_p[x]$. Therefore, each root in $\mathbb{F}_p$ of $\tilde{f}_j(x) = f_j(x) - w_j$ divides $\gcd(\pi_p, \tilde{f}_j)$. If a linear polynomial $x - x_0$ divides the polynomial

$$\gcd\left(\gcd(\pi_p, \tilde{f}_{j_1}), \dots, \gcd(\pi_p, \tilde{f}_{j_n})\right),$$

then x_0 is a solution to the original CICO problem.

The root-finding stage can then be further divided into two sub-stages:

1. **Compute** $\tilde{\pi}_{p,j} = \pi_p \bmod \tilde{f}_j$. Since direct computation of π_p is impractical due to its large degree, we reduce the problem of computing $\gcd(\pi_p, \tilde{f}_j)$ to the equivalent task of computing $\gcd(\tilde{\pi}_{p,j}, \tilde{f}_j)$, where $\tilde{\pi}_{p,j} = \pi_p \bmod \tilde{f}_j$ has degree less than $\deg(\tilde{f}_j)$. To achieve this, we first compute $x^p \bmod \tilde{f}_j$ using a square-and-multiply approach based on the binary representation of p . We then subtract $x \bmod \tilde{f}_j$ from this result.
2. **GCD Computations.** We compute $g_j = \gcd(\tilde{\pi}_{p,j}, \tilde{f}_j)$. Assuming that POSEIDON behaves similarly to a random permutation, the univariate transformation from the first step implies that each polynomial g_j is expected to have an average degree close to 1. We then compute the polynomial $g = \gcd(g_1, \dots, g_n)$, and select any root of g as a solution to the original CICO problem.

The complexity of solving the CICO problem using the outlined root-finding technique is primarily determined by polynomial arithmetic operations. The most computationally demanding step is the Root-finding, specifically the computation of $\tilde{\pi}_{p,j} = \pi_p \bmod \tilde{f}_j$, which involves exponentiation modulo a polynomial of degree $d = \deg(\tilde{f}_j)$. Applying a square-and-multiply method combined with fast polynomial multiplication, this step has a time complexity of approximately $\mathcal{O}(M(d) \log p)$, where $M(d)$ denotes the cost of multiplying two polynomials of degree less than d over $\mathbb{F}_p$.

The subsequent polynomial GCD computations have complexity bounded by $\mathcal{O}(M(d) \log d)$, which is generally less costly. Recall that $M(d) \in \mathcal{O}(d \log d \log \log d)$ with state-of-the-art polynomial multiplication algorithms. Therefore, the root-finding process scales quasi-linearly with the polynomial degree, with the modular exponentiation step representing the computational bottleneck.

4 Root-finding Based on the Graeffe Transform

The root-finding strategy outlined in [Subsection 3.1](#) is a well-known method for identifying roots of polynomials over finite fields, with its origins tracing back to Berlekamp’s polynomial factorization algorithm. However, this algorithm is generic in nature and does not rely on any specific structural properties of the fields in which the polynomials are defined to improve computational efficiency.

In this section, we will introduce the Graeffe transform and examine its various properties. These properties will subsequently be used to accelerate polynomial root-finding over NTT-friendly fields—namely, that is finite fields $\mathbb{F}_p$ in which $p - 1$ is divisible by a large power of a small prime, such as 2.

Definition 1. *The Graeffe transform of order $r > 0$ of a monic polynomial $P \in \mathbb{F}_p[x]$ of degree d is defined as the unique monic polynomial $G(P) \in \mathbb{F}_p[x]$ of degree d satisfying*

$$G_r(P)(x^r) = (-1)^{(r-1)d} \cdot P(x) \cdot P(\omega_r \cdot x) \cdot \dots \cdot P(\omega_r^{r-1} \cdot x)$$

with $\omega_r \in \mathbb{F}$ a primitive r -th root of unity.

We are interested in exploring two properties of Graeffe transform: composition and roots’ exponentiations.

4.1 Composition of Graeffe Transforms

A consequence of [Definition 1](#) is that for integers $r, s > 0$, the transforms satisfies

$$G_{rs}(P) = (G_r \circ G_s)(P) = (G_s \circ G_r)(P).$$

This property allows efficient computation of Graeffe transforms of order r^k , by iteratively applying k times a transform of order r .

In practice, Graeffe transforms of order r , with r prime, are computed using fast NTT-based polynomial multiplication. We recall that a generic product polynomial $f = f_0(x) \dots f_{r-1}(x) \in \mathbb{F}_p[x]$ can be efficiently computed in $O(n \log n)$ operations as

$$f = NTT_n^{-1} (NTT_n(f_0) \circ NTT_n(f_1) \circ \dots \circ NTT_n(f_{r-1}))$$

where $n \geq 1 + \sum \deg(f_i)$, $\mathbb{F}_p$ admits an n -th primitive root of unity, NTT_n and NTT_n^{-1} denote the forward and inverse NTT of size n and $\circ$ denotes element-wise product of the NTT results.

It follows that naively computing a Graeffe transform of order r^k as

$$G_{r^k}(P)(x^{r^k}) = (-1)^{(r-1)d} \cdot NTT_n^{-1} \left(NTT_n(P(x)) \circ NTT_n(P(\xi x)) \circ \dots \circ NTT_n(P(\xi^{r^k-1} x)) \right)$$

with $\omega_{r^k} \in \mathbb{F}_p$ a primitive r^k root of unity, requires at least $O(r^k d \log(r^k d))$ operations, a much worse complexity compared to the $O(krd \log(rd))$ operations required by iteratively applying k times a Graeffe transform of order r .

These observations imply that computing Graeffe’s transforms is particularly efficient over finite fields $\mathbb{F}_p$ with $p - 1$ divided by a large power of a small prime such as 2, in which cases the field is said to be NTT-friendly.

4.2 Roots exponentiation

From [Definition 1](#), if $\alpha \in \mathbb{F}_p$ is a root of $P(x)$, then $(x - \omega_r^{r-i} \alpha)$ divides $P(\omega_r^i x)$ for all $i \in [0, r - 1]$. Therefore, $\prod_{i=0}^{r-1} (x - \omega_r^{r-i} \alpha) = x^r - \alpha^r$ divides $G_r(P)(x^r)$, and thus α^r is a root of $G_r(P)(x)$.

When working over a finite field $\mathbb{F}_p$ with $p - 1 = r \cdot s$ and $\gcd(r, s) = 1$, the Graeffe transform of order r of a polynomial $P(x)$ maps non-zero roots α of $P(x)$ to non-zero roots α^r of $G_r(P)(x)$, with the latter lying in the subgroup of order s of $\mathbb{F}_p^\times$. This implies that $\alpha^r = \omega_s^i$ for some $i \in [0, s - 1]$ and $\omega_s \in \mathbb{F}_p$ primitive s -th root of unity.

To efficiently find non-zero roots of $G_r(P)(x)$, we can proceed as follows:

- If $\deg G_r(P)(x) \leq s$, we compute in $O(s \log s)$ operations $NTT_s(G_r(P))$, then perform an $O(s)$ search for zero entries in the NTT result. Indeed, computing $NTT_s(G_r(P))$ corresponds to evaluating $G_r(P)(x)$ at all powers of a primitive s -th root of unity ω_s . Thus, a zero entry at index i indicates that ω_s^i is a root of $G_r(P)(x)$.
- If $\deg G_r(P)(x) > s$, we compute $\tilde{g}(x) = G_r(P)(x) \pmod{x^s - 1}$ and we proceed as above to find a root of $\tilde{g}(x)$. Since $(x - \omega_s^i)$ divides $x^s - 1$ for all $i \in [0, s-1]$, whenever $(x - \omega_s^i)$ divides $G_r(P)(x)$, then $(x - \omega_s^i)$ must necessarily divide $\tilde{g}(x)$, hence ω_s^i is one of its root.

4.3 Tangent Graeffe Transforms

The Graeffe transform, by itself, does not allow for the direct computation of the roots of a polynomial $P(x) \in \mathbb{F}_p[x]$. However, a method known as the *Tangent Graeffe Transform* enables the extraction of such roots by operating over the ring of dual numbers. In this Section, we will briefly sketch this method.

Let ε be a formal indeterminate such that $\varepsilon^2 = 0$. Elements of the quotient ring $\mathbb{F}_p[\varepsilon]/(\varepsilon^2)$ are referred to as *tangent numbers*. These take the form $a + b\varepsilon$, where $a, b \in \mathbb{F}_p$, and the basic arithmetic operations are defined as follows:

$$\begin{aligned} (a + b\varepsilon) \pm (c + d\varepsilon) &= (a \pm c) + (b \pm d)\varepsilon, \\ (a + b\varepsilon)(c + d\varepsilon) &= ac + (ad + bc)\varepsilon. \end{aligned}$$

Let $P \in \mathbb{F}_p[x]$ be a monic polynomial of degree d that splits completely over $\mathbb{F}_p$ as $P(x) = \prod_{i=1}^d (x - \alpha_i)$, where the roots $\alpha_1, \dots, \alpha_d \in \mathbb{F}_p$ are pairwise distinct. Define the *tangent deformation* $\tilde{P}(x) := P(x + \varepsilon)$. Using Taylor expansion, we obtain:

$$\tilde{P}(x) = P(x) + P'(x)\varepsilon = \prod_{i=1}^d (x - (\alpha_i - \varepsilon)).$$

The Graeffe transform of order $r \geq 2$, previously defined over $\mathbb{F}_p$, extends naturally to polynomials with coefficients in $\mathbb{F}_p[\varepsilon]$. We define the *tangent Graeffe transform* of order r as $G_r(\tilde{P})$, satisfying

$$G_r(\tilde{P})(x) = \prod_{i=1}^d (x - (\alpha_i - \varepsilon))^r,$$

where each factor expands as

$$(\alpha_i - \varepsilon)^r = \alpha_i^r - r\alpha_i^{r-1}\varepsilon \quad \text{for } i = 1, \dots, d.$$

Assume we have an efficient method for computing the roots $\alpha_1^r, \dots, \alpha_d^r$ of $G_r(P)$ (see [Subsection 4.2](#)). Then $G_r(\tilde{P})$ may be written as:

$$G_r(\tilde{P})(x) = A(x) + B(x)\varepsilon,$$

for some polynomial $B \in \mathbb{F}_p[x]$ and $A = G_r(P)$. For each root α_i^r of A , we evaluate:

$$G_r(\tilde{P})(\alpha_i^r - r\alpha_i^{r-1}\varepsilon) = A(\alpha_i^r) + (B(\alpha_i^r) - A'(\alpha_i^r) \cdot r\alpha_i^{r-1})\varepsilon = 0$$

Since α_i^r is a root of A , the above simplifies to:

$$(B(\alpha_i^r) - A'(\alpha_i^r) \cdot r\alpha_i^{r-1})\varepsilon = 0.$$

Therefore, if α_i^r is a simple root of A , it follows that $r\alpha_i^{r-1} = \frac{B(\alpha_i^r)}{A'(\alpha_i^r)}$. Provided that $\alpha_i^r \neq 0$, we can then recover the original root α_i of $P(x)$ as:

$$\alpha_i = \frac{r\alpha_i^r}{r\alpha_i^{r-1}} = r\alpha_i^r \cdot \frac{A'(\alpha_i^r)}{B(\alpha_i^r)}.$$

4.4 Heuristic Randomized Algorithm for Root Finding over a NTT-friendly Fields

The Tangent Graeffe Method outlined in the previous Section has been extensively analyzed in [8], where the authors formalized an heuristic randomized algorithm for finding all roots of a polynomial over NTT-friendly fields. Their method has been successfully implemented in [9] and [10], where, for the prime $p = 5 \cdot 2^{55} + 1$, it was used to compute the roots of polynomials of degree up to 10^9 . In this Section we will briefly review their algorithm as originally introduced in [8].

Let $\mathbb{F}_p$ be a finite field, where p is a prime of the form $p = \sigma \cdot 2^m + 1$ for some small σ . Suppose that $\beta \in \mathbb{F}_p$ is a primitive element of order $p - 1$ in the multiplicative group of $\mathbb{F}_p$.

Let $P = (x - \alpha_1) \cdots (x - \alpha_d) \in \mathbb{F}_p[x]$ be as defined in the previous subsection. The tangent Graeffe method can be used to efficiently compute those α_k for which α_k^r is a simple root of $G_r(P)$. In order to ensure a sufficient number of such roots, the polynomial $P(x)$ is first replaced by $P(x + \tau)$.

Let r be the largest power of two such that $r \leq \frac{p-1}{4d}$, and define $s = \frac{p-1}{r}$. By construction, $s = O(d)$. The roots $\alpha_1^r, \dots, \alpha_d^r$ of $G_r(P)$ are s -th roots of unity and lie in the set $\{1, \omega, \dots, \omega^{s-1}\}$, where $\omega = \beta^r$. These roots can be determined by evaluating $G_r(P)$ at ω^i for $i = 0, \dots, s - 1$. Since $s = O(d)$, this evaluation can be performed efficiently using a discrete Fourier transform. Combined with the tangent Graeffe method described in the previous subsection, this yields the following probabilistic algorithm for root finding, along with its complexity expressed in $\mathcal{O}$ notation:

Algorithm 1: Randomized Root Finding Algorithm over NTT-Friendly Fields [8]

Input: $P \in \mathbb{F}_p[x]$ of degree d and only order one factors, $p = \sigma 2^m + 1$
Output: The set $\{\alpha_1, \dots, \alpha_d\}$ of roots of P
Data: Time complexities for the main steps are indicated on the right

```

1 if  $d = 0$  then
2   return  $\emptyset$ 
3 Let  $r = 2^N \in \mathbb{N}$  be the largest such that  $r \leq \frac{p-1}{4d}$ , and let  $s := \frac{p-1}{r}$ ;
4 Pick  $\tau \in \mathbb{F}_p$  at random and compute  $P^* := P(x + \tau) \in \mathbb{F}_p[x]$   $\mathcal{O}(M(d))$ ;
5 Compute  $\tilde{P}(x) := P^*(x + \epsilon) = P^*(x) + P^*(x)' \epsilon \in (\mathbb{F}_p[\epsilon]/(\epsilon^2))[x]$ ;
6 For  $i = 1, \dots, N$ , set  $\tilde{P} := G_2(\tilde{P}) \in (\mathbb{F}_p[\epsilon]/(\epsilon^2))[x]$   $\mathcal{O}(M(d) \log p / s)$ ;
7 Let  $\omega \in \mathbb{F}_p^*$  be of order  $s$ , and write  $\tilde{P} = A + B\epsilon$ ;
8 Compute  $A(\omega^i), A'(\omega^i), B(\omega^i)$  for  $i = 0, \dots, s - 1$   $\mathcal{O}(M(s))$ ;
9 if  $P(\tau) = 0$  then
10   Set  $S := \{\tau\}$ 
11 else
12   Set  $S := \emptyset$ 
13 For  $\beta \in \{1, \omega, \dots, \omega^{s-1}\}$ ;
14 if  $A(\beta) = 0$  and  $A'(\beta) \neq 0$  then
15   Set  $S := S \cup \left\{ r\beta \frac{A'(\beta)}{B(\beta)} + \tau \right\}$ 
16 Compute  $Q := \prod_{\alpha \in S} (x - \alpha)$   $\mathcal{O}(M(d) \log d)$ ;
17 Compute  $R := \frac{\tilde{P}}{Q}$   $\mathcal{O}(M(d))$ ;
18 Recursively determine the set of roots  $S'$  of  $R$  and return  $S \cup S'$ 

```

The computational complexity of the presented root-finding algorithm is dominated by polynomial arithmetic over the ring $\mathbb{F}_p[\epsilon]/(\epsilon^2)$. The key cost arises from the repeated application of the operator G_2 , which involves $N = \frac{r}{2}$ iterations and contributes a time complexity of approximately $\mathcal{O}\left(M(d) \frac{\log p}{s}\right)$, where $M(d)$ denotes the complexity of multiplying polynomials of degree at most d over $\mathbb{F}_p$. Evaluating the polynomials A , A' , and B at s points has complexity $\mathcal{O}(M(s))$, while polynomial division and multiplication steps incur complexities bounded by $\mathcal{O}(M(d) \log d)$ and $\mathcal{O}(M(d))$, respectively. Given that $M(d) \in \mathcal{O}(d \log d \log \log d)$ using fast multiplication techniques,

the overall root-finding procedure runs quasi-linearly in d , with the exponentiation steps via G_2 dominating the runtime.

5 Attacking Poseidon and Poseidon2

5.1 The Skip-Round Attack

Bariant et al. [2] introduced a univariate transformation that simplifies solving the CICO problem for POSEIDON, when exactly one input state element is constrained. Without loss of generality, and for ease of exposition, we assume that this constraint sets the last input and output indices to 0.

Their core strategy decomposes the full POSEIDON permutation P into two sub-permutations, $P = P_2 \circ P_1$. This decomposition allows one to solve a simplified modified CICO problem for P_2 whose solution can be efficiently mapped to a solution to the original CICO problem for P .

Remarkably, their approach requires the application of a univariate transformation to P_2 (and consequently to P), constraining the inputs to the particular form

$$(A_0 \cdot x + B_0, A_1 \cdot x + B_1, \dots, A_{T-1} \cdot x + B_{T-1})$$

for certain efficiently computable constants $A_0, B_0, \dots, A_{T-1}, B_{T-1} \in \mathbb{F}_p$. These constants satisfy

$$P_1^{-1}(A_0 \cdot x + B_0, A_1 \cdot x + B_1, \dots, A_{T-1} \cdot x + B_{T-1}) = (v_0, \dots, v_{T-2}, 0)$$

for every possible choice of $x \in \mathbb{F}_p$.³

Consequently, if x_0 solves the modified CICO problem given by

$$P_2(A_0 \cdot x + B_0, A_1 \cdot x + B_1, \dots, A_{T-1} \cdot x + B_{T-1}) = (w_0, \dots, w_{T-2}, 0),$$

then the vector

$$(v_0, \dots, v_{T-2}, 0) = P_1^{-1}(A_0 \cdot x + B_0, A_1 \cdot x + B_1, \dots, A_{T-1} \cdot x + B_{T-1})$$

solves the original CICO problem

$$P(v_0, \dots, v_{T-2}, 0) = (w_0, \dots, w_{T-2}, 0)$$

The 2-Round Attack for Poseidon In the case of POSEIDON, we define the permutation P_1 as comprising the first two full rounds, omitting the second mixing layer. The subsequent permutation P_2 includes this omitted mixing layer as well as all remaining rounds.

The univariate representation of P_2 yields output polynomials $f_j(x)$ whose degrees are lower than those obtained from the full permutation P . This reduction in degree is due to the absence of two S-Boxes in P_2 that are otherwise present in the complete permutation. As a result, we obtain the bound $\deg(f_j) \leq d^{R_F + R_P - 2}$ which significantly simplifies both interpolation and root-finding when solving the CICO problem.

The 1-Round Attack for Poseidon2 While the authors of [2] did not explicitly apply their technique to POSEIDON2, the method can be adapted to skip one full round. For POSEIDON2, we define the permutation P_1 to consist of the initial application of the external matrix M_ϵ , followed by the first full round, excluding the final mixing layer M_ϵ . The permutation P_2 then comprises the mixing layer of the first full round followed by all subsequent rounds.

The corresponding univariate transformation of P_2 then results in output polynomials $f_j(x)$ satisfying the bound $\deg(f_j) \leq d^{R_F + R_P - 1}$.

³ Only one input entry can be constrained with this technique. Without loss of generality, we constrain the last entry to 0.

5.2 Accelerate Root-finding with Graeffe's Transform

In the previous section, we saw how the Graeffe Transform and the Tangent Graeffe method can be used to find roots of a polynomial over $\mathbb{F}_p$. However, as discussed in [Section 3](#), solving the CICO-1 problem for the univariate transformation requires finding a single root, thus we can optimize [algorithm 1](#) for the specific instances of POSEIDON and POSEIDON2 we want to attack.

In this section, we describe the modifications made to obtain a more efficient algorithm tailored to finding a single root (if one exists) of a polynomial $P(x) \in \mathbb{F}_p[x]$. We will first present it, and then examine its components in detail:

Algorithm 2: Single Root Finding Algorithm

Input: $P \in \mathbb{F}_p[x]$ of degree d ; $r, s \in \mathbb{N}$ so that $p = rs + 1$ and $r = 2^k$ for a certain $k \in \mathbb{N}$, ω_r and ω_s primitive r -th and s -th, respectively, roots of unity of $\mathbb{F}_p$.

Output: An $\alpha \in \mathbb{F}_p$ such that $P(\alpha) = 0$, if one exists.

```

1 if  $d = 0$  then
2   return  $\emptyset$ 
3 if  $P(0) = 0$  then
4   return 0
5 Set  $G(x) = P(x)$ . For  $i = 0, \dots, k - 1$ , compute  $G(x) = G_2(G)(x)$      $\mathcal{O}(M(d) \log r)$ ;
6 Set  $\tilde{P}(x) = G(x) \pmod{x^s - 1}$ ;
7 Compute the vector  $V = NTT_s(\tilde{P})$  using  $\omega_s \in \mathbb{F}_p$      $\mathcal{O}(s \log s)$ ;
8 for  $i = 0, \dots, s - 1$  do
9   if  $V[i] = 0$  then
10    Set  $\beta = \omega_s^i$ ;
11    break
12  else
13    return  $\emptyset$ 
14 For  $i = 0, \dots, k - 1$ , compute  $\beta = \sqrt{\beta}$ ;
15 for  $i = 0, \dots, r - 1$  do
16   Set  $\alpha = \omega_r^i \cdot \beta$ ;
17   if  $P(\alpha) = 0$  then
18     return  $\alpha$ 
19 return  $\emptyset$ 

```

Since we are interested in finding a single root of $P(x)$, for the sake of exposition we will assume that it has at most one root and that this is non-zero.

The main idea underlying the algorithm is that when working over a finite field $\mathbb{F}_p$ where $p - 1 = r \cdot s = 2^k \cdot s$, we can iteratively apply the efficient Graeffe transform of order 2 exactly k times to the polynomial $P(x)$ ([line 5](#)). This process effectively “pushes” the root α of P to the root α^r of the transformed polynomial $G_r(P)$, which lies in the multiplicative subgroup of order s of $\mathbb{F}_p$.

Since all roots of $G_r(P)$ must have order dividing s , they are also roots of $x^s - 1$. Therefore, they are also roots of the polynomial $\tilde{P}(x) = G_r(P)(x) \pmod{x^s - 1}$, which has degree less than s ([line 6](#)). We note that this step can be computed in $\max(d - s, 0)$ operations by folding the coefficients of $G(x)$ according to the relation $x^s = 1 \pmod{x^s - 1}$.

At this stage, we apply NTT_s to $\tilde{P}$ in order to evaluate it over the full subgroup of order s . This allows us to efficiently locate its root α^r , if one exists ([line 7](#), [line 8](#)).

Once a root $\beta = \omega_s^i = \alpha^r$ of $\tilde{P}(x)$ is found, we can recover the original root α of $P(x)$ as follows:

- Extract an r -th root $\beta = \sqrt[r]{\beta} \in \mathbb{F}_p$ by iteratively computing k square roots ([line 14](#)).

- For all $i \in [0, r-1]$, check whether $P(\omega_r^i \cdot \beta) = 0$ (line 15). This works because the values $\omega_r^i \cdot \beta$ represent all roots of the polynomial $x^r - \alpha^r$, and thus the true root α must equal one of these elements for some exponent i .

We note that the complexity of the final loop (line 15) is, in principle, $O(dr)$ for an arbitrary polynomial $P(x) \in \mathbb{F}_p[x]$ of degree d . However, in the context of attacking Poseidon, the polynomial $P(x)$ corresponds to the univariate representation of the permutation P_2 , as defined in Subsection 5.1. Since the evaluation of P_2 can be performed with a constant number of operations once the targeted Poseidon instance is fixed, this approach is more efficient than evaluating the univariate polynomial directly, which would otherwise incur a cost of $O(d)$ operations per evaluation. In practical attacks, the last loop is then performed by evaluating the permutation P_2 over an input state evaluated at α , thus costing $O(r)$ operations overall.

When applied to Poseidon and Poseidon2 permutations, the total cost of the optimize single root finding algorithm is then $O(M(d) \log r + s \log s + r)$.

Large Input Polynomials. We recall that the degree d of the Graeffe transform $G(x)$ is equal to the degree of the input polynomial $P(x)$. Since, in line 6, $G(x)$ will be reduced modulo $x^s - 1$, it is advantageous—particularly when $d \gg s$ —to gradually reduce the size of the intermediate Graeffe transforms down to s during the execution of line 5. This strategy helps to reduce the memory footprint of the algorithm without affecting correctness.

The key observation is that if $\alpha \neq 0$ is a root of a polynomial $P(x)$, then α^{2^i} is a root of the Graeffe transform $G_{2^i}(P)$. Since this value lies in the multiplicative subgroup of order $\frac{p-1}{2^i}$ in $\mathbb{F}_p^\times$, it follows that α^{2^i} is a root of the polynomial $x^{\frac{p-1}{2^i}} - 1$, as well as of the reduced polynomial

$$\tilde{G}_{2^i}(x) = G_{2^i}(x) \pmod{x^{\frac{p-1}{2^i}} - 1}$$

whose degree satisfies

$$\deg \tilde{G}_{2^i} = \min \left(\deg(G_{2^i}), \frac{p-1}{2^i} - 1 \right)$$

Thus, as soon as $d = \deg(G_{2^i}) > \frac{p-1}{2^i} - 1$, that is, starting from iteration

$$i = \lceil \log_2 \left(\frac{p-1}{d} \right) \rceil$$

we can begin reducing the size of the transformed polynomials at each step of line 5 by halving, continuing until their degree falls below the threshold s .

Large NTT-Friendly Fields. When the field is small, performing the NTT of size s at line 7 poses no practical difficulties. However, for large fields—such as BLS12-381—computing such an NTT is infeasible due to memory and computational constraints.

To understand the necessity of computing such large NTTs, we observe that the goal is to find a root in the subgroup of order s of $\mathbb{F}_p^\times$ for the Graeffe transform G of order r applied to $P(x)$ (since $s \gg r$, we have $\tilde{P} = G$ in line 6).

As a result, we may alternatively employ the classical root-finding strategy based on Berlekamp’s polynomial factorization algorithm, as described in Subsection 3.1, but with a key adaptation: we compute $\gcd(G(x), x^s - 1 \bmod G(x))$ instead of $\gcd(G(x), x^p - x \bmod G(x))$, thus saving $\log r$ exponentiation and modulo reduction steps. Once a root α^r to G is found, we then proceed from line 15.

5.3 Breaking Reduced-Round Instances of Poseidon and Poseidon2

We now demonstrate how the techniques developed in the previous sections can be applied to break reduced-round instances of the POSEIDON and POSEIDON2 permutations over prime fields. Our focus is on instances defined over NTT-friendly fields $\mathbb{F}_p$, where $p = \sigma \cdot 2^m + 1$ and efficient root-finding techniques are applicable. The reduced-round instances analyzed here originate from the Ethereum Foundation bug bounty program, as detailed on the official POSEIDON initiative website [1]. We apply our method to instances with 64-bit and 256-bit state sizes. In the 64-bit setting, we employ the Graeffe-based root-finding approach described in Section 4, while for the 256-bit instances, we rely on the more traditional algebraic method outlined in Section 3. In both settings, we demonstrate that our root-finding strategy can recover a preimage (or equivalently, solve the CICO-1 problem).

The remainder of this Section details the specific parameters used, the reduction in rounds, and the computational results achieved by our attack. These primes were chosen for their NTT-friendly properties, enabling efficient polynomial operations and root-finding algorithms in our attack. The factorizations of $p - 1$ support the use of large-order multiplicative subgroups, which are essential for the Graeffe transform techniques discussed in Section 4.

Attack running times were measured on a machine equipped with an AMD EPYC 9374F 32-core processor, 1.1 TB of RAM, and 8 NVIDIA L4 GPUs, each with 23 GiB of memory. The AWS cloud instance closest to this hardware configuration is the `g6.48xlarge`, featuring 192 vCPUs, 768 GiB of memory, and 8 NVIDIA L4 GPUs, priced at \$13.35/h on-demand.

Poseidon-256 The POSEIDON bounty instances for the 256-bit state size are defined over the scalar field $\mathbb{F}_p$ of the BLS12-381 elliptic curve [3], where

$$p - 1 = 2^{32} \cdot 3 \cdot 11 \cdot 19 \cdot 10177 \cdot 125527 \cdot 859267 \cdot 906349^2 \cdot 2508409 \cdot 2529403 \cdot 52437899 \cdot 254760293^2.$$

We examine the computational cost and resource demands of solving each bounty instance detailed in Table 1.

Instance	Field	Bits	d	T	R_F	R_P	κ	CI	CO	GPU	Interp.	Root	Find.
P_6_8						8	24			No	1m 20s	3d 7h 17m	*
P_6_8_GPU	BLS12-381	256	5	3	6	8	24	1	1	Yes	45s	9h 17m	*
P_6_9						9	28			Yes	6m 10s	2d 18h 10m	

*Our solution was not the first on this instance (P_6_8).

Table 1. Parameters and attack cost summary for POSEIDON reduced-round instances at security level κ . Running time measured on a machine equipped with an AMD EPYC 9374F 32-core processor, 1.1 TB of RAM, and 8 NVIDIA L4 GPUs, each with 23 GiB of memory.

Since each scalar in the BLS12-381 implementation requires 32 bytes of storage, applying the 2-round attack to each bounty instance involves interpolating a univariate polynomial of degree $5^{(R_F + R_P - 2)}$. Specifically, the polynomials interpolating the instances 6_8 and 6_9 require approximately 7.3 GiB and 36.4 GiB of memory, respectively.

To benefit from the efficiency of radix-2 transforms, we pad polynomials with zeros to the nearest power-of-two length before applying the NTT. Consequently, the interpolated polynomials are represented as vectors of size 2^{28} and 2^{31} for instances 6_8 and 6_9, respectively, each entry occupying 32 bytes. All implemented polynomial arithmetic primitives are optimized to use NTTs whose sizes are at most twice the size of their input polynomials. Therefore, for these bounty instances, we will handle multiple NTTs of sizes 2^{29} and 2^{32} , respectively, reaching the finite field supported 2-radix NTT size limit of 2^{32} for the application of the fast 2-radix algorithm.

Instance	Field	Bits	d	T	R_F	R_P	κ	CI	CO	GPU	Interp.	Root	Find.
P2_6.7	Goldilocks	64	7	8	6	7	24	1	1	Yes	20m	9h 38m [*]	

^{*}Our solution was not the first submitted for this instance (P2_6.7).

Table 2. Parameters and attack cost summary for POSEIDON2 reduced-round instances. Running time measured on a machine equipped with an AMD EPYC 9374F 32-core processor, 1.1 TB of RAM, and 8 NVIDIA L4 GPUs, each with 23 GiB of memory.

Unfortunately, execution-time measurements for the NTL-based GCD computations are no longer available. However, this step required a few hours of computation on a single CPU core for the 6.8 instance and around 3 days for 6.9. Since the GCD step does not require GPU acceleration, we estimate its computational cost independently, using a CPU-only AWS instance (`r5.24xlarge`), featuring 96 vCPUs, 768GiB RAM, priced at \$6.05/h on-demand. We remark that enabling parallel execution or optimizing further this GCD implementation could significantly reduce this machine cost overhead.

This results in a total estimated attack cost of:

- **Instance P_6.8: \$185**, computed as 9h17m at \$13.35/h plus an additional 10h at \$6.05/h. The bounty prize was set at **\$4000**.
- **Instance P_6.9: \$1101**, computed as 2d18h10m at \$13.35/h plus an additional 36h at \$6.05/h. The bounty prize was set at **\$6000**.

We note that these attacks were performed using the classical strategy described in [Subsection 3.1](#), without employing the potential optimizations afforded by the Graeffe transform, as outlined in [Section 19](#). As a result, the reported cost and time estimates may be conservative. Investigating the potential improvements achievable through the use of Graeffe-based methods is left as future work.

Poseidon-64 The POSEIDON2 bounty instances for 64-bit state size are defined over the Goldilocks prime field $\mathbb{F}_p$, where

$$p - 1 = 2^{32} \cdot 3 \cdot 5 \cdot 17 \cdot 257 \cdot 65537.$$

As presented in [\[11\]](#), our attacks targeted the CICO-1 POSEIDON2 bounty instance at the 24-bits security level, proposed as part of the Ethereum Foundation’s bounty program. In [Table 2](#) we summarize the relevant parameters for this instance, including its S-box degree d , number of rounds, and constraint configuration, along with the measured running times of our attack on high-performance GPU-equipped hardware.

In contrast to the Poseidon-256, only a single round can be skipped in POSEIDON2 instances (see [Subsection 5.1](#)), and the higher S-Box degree further increases the memory requirements for polynomial arithmetic. The univariate transformation of the permutation P_2 yields a polynomial of degree $7^{6+7-1} \approx 2^{33.7}$, which significantly exceeds the maximum size limit of a radix-2 NTT over the field, capped at 2^{32} . As a consequence, it becomes necessary to employ mixed-radix NTTs capable of handling sizes up to $q \cdot 2^{32}$, with q a small integer, allowing the efficient radix-2 NTT algorithm to be used for the radix-2 component of the computation.

However, q must be chosen large enough to accommodate all NTT sizes required at various stages of [algorithm 2](#), including the Graeffe transforms, the NTT of size s (we employ Bluestein’s Algorithm to deal with NTT sizes not dividing $q \cdot 2^{32}$), and any corresponding padded sizes needed during computation (for example, in order to perform NTT of size s using Bluestein’s algorithm, we must support sizes exceeding $2s$). In order to attack the $P2_6_7$ instance, we chose $q = 15$.

Another important consideration is the choice of parameters $r = 2^k$ and s such that $r \cdot s = p - 1$. This selection is influenced by the resulting degree d of the attacked polynomial, since

each Graeffe transform of order 2 requires three NTTs of size d , amounting to a total of $3k$ NTTs. This cumulative cost must be balanced against the single NTT of size s , which can be treated as a constant overhead provided it falls within the limits supported by mixed-radix NTT implementations.

In the attack on the *P2_6.7* instance, we selected $r = 2^{30}$ and $s = 4(2^{32} - 1)$, resulting in an approximate runtime of 17 minutes for each Graeffe transform of order 2, and about one hour for the final Bluestein’s NTT of size s . At the time of this attack, the Graeffe transform implementation was sub-optimal, as it followed the original definition: computing the product of two polynomials of degree d , thereby requiring three NTTs of size $2d$. However, by noting that the Graeffe transform of order 2 satisfies the identity

$$G_2(P) = (-1)^d (P_e(x)^2 - xP_o(x)^2)$$

where $P(x) = P_e(x^2) + x \cdot P_o(x^2)$, we can reduce the computational cost by performing only three NTTs of size d instead. As a result, we expect the total runtime of the attack to be reduced by approximately half compared to the time reported in [Table 2](#). Verifying this performance improvement is left as future work.

This results in a total estimated attack cost of:

- **Instance P2_6.8: \$133**, computed as 9h58m at \$13.35/h. The bounty prize was set at **\$4000**.

Conclusion

We have presented a root-finding strategy based on the Graeffe transform, tailored to instances of the POSEIDON and POSEIDON2 permutations instantiated over NTT-friendly prime fields. By exploiting the specific algebraic structure of these constructions, the proposed method streamlines the root recovery process in interpolation attacks and demonstrates its practical effectiveness by successfully solving the CICO-1 problem for reduced-round instances proposed by the Ethereum Foundation within the Poseidon Cryptanalysis Initiative.

This work underscores the importance of comprehensive security evaluations for cryptographic permutations, particularly when deployed over structured fields that may enable specialized algebraic attacks. Future research directions include extending these techniques to broader parameter ranges and to other permutation-based primitives, with the aim of further understanding their security properties.

References

1. Poseidon initiative. <https://www.poseidon-initiative.info/>, accessed: 2025-05-16 [11](#)
2. Bariant, A., Bouvier, C., Leurent, G., Perrin, L.: Algebraic attacks against some arithmetization-oriented primitives. *IACR Transactions on Symmetric Cryptology* **2022**(3), 73–101 (9 2022). <https://doi.org/10.46586/tosc.v2022.i3.73-101>, <https://tosc.iacr.org/index.php/ToSC/article/view/9850> [2](#), [8](#)
3. Bowe, S.: BLS12-381: New zk-SNARK Elliptic Curve Construction (2017), <https://electriccoin.co/blog/new-snark-curve/> [11](#)
4. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schafneger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: Bailey, M., Greenstadt, R. (eds.) *USENIX Security 2021*. pp. 519–535. USENIX Association (Aug 2021) [1](#)
5. Grassi, L., Khovratovich, D., Schafneger, M.: Poseidon2: A faster version of the poseidon hash function. In: El Mrabet, N., De Feo, L., Duquesne, S. (eds.) *AFRICACRYPT 23*. LNCS, vol. 14064, pp. 177–203. Springer, Cham (Jul 2023). https://doi.org/10.1007/978-3-031-37679-5_8 [1](#)
6. Grassi, L., Lüftenegger, R., Rechberger, C., Rotaru, D., Schafneger, M.: On a generalization of substitution-permutation networks: The HADES design strategy. In: Canteaut, A., Ishai, Y. (eds.) *EUROCRYPT 2020, Part II*. LNCS, vol. 12106, pp. 674–704. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45724-2_23 [1](#)

7. Grassi, L., Rechberger, C., Schofnegger, M., Khovratovich, D.: Survey of cryptanalytic attacks on poseidon and poseidon2 (2025), available at: https://drive.google.com/file/d/1bqmIk5I8s-4S9TQJ00xk26fnSSU0q_Hx/view 1
8. Grenet, B., van der Hoeven, J., Lecerf, G.: Randomized root finding over finite fft-fields using tangent graeffe transforms. In: Proceedings of the 2015 ACM International Symposium on Symbolic and Algebraic Computation. p. 197–204. ISSAC '15, Association for Computing Machinery, New York, NY, USA (2015). <https://doi.org/10.1145/2755996.2756647>, <https://doi.org/10.1145/2755996.2756647> 7
9. van der Hoeven, J., Monagan, M.: Implementing the tangent graeffe root finding method. In: Bigatti, A.M., Carette, J., Davenport, J.H., Joswig, M., de Wolff, T. (eds.) Mathematical Software – ICMS 2020. pp. 482–492. Springer International Publishing, Cham (2020) 7
10. van der Hoeven, J., Monagan, M.: Computing one billion roots using the tangent graeffe method. ACM Commun. Comput. Algebra **54**(3), 65–85 (Mar 2021). <https://doi.org/10.1145/3457341.3457342>, <https://doi.org/10.1145/3457341.3457342> 7
11. Sanso, A., Vitto, G.: Poseidon over finite fft-fields: Leveraging graeffe transform ftw. Talk at Algebraic Hash Cryptanalysis Days, Eurocrypt Affiliated Event (2025), madrid, Spain 12